

1. Perceptron to Classify Flipkart Reviews

Problem

Classify a Flipkart review as:

- **Positive = 1**
- **Negative = 0**

Features:

- Review Length
- Number of Exclamation Marks

```
import numpy as np
```

```
# -----  
# Training Dataset  
# [Review Length, Exclamation Marks]  
# -----
```

```
X = np.array([  
    [90, 6],    # Positive  
    [80, 5],    # Positive  
    [70, 4],    # Positive  
    [25, 0],    # Negative  
    [30, 1],    # Negative  
    [40, 0]     # Negative  
])
```

```
y = np.array([1,1,1,0,0,0])
```

```
# Normalize  
X = X / np.max(X, axis=0)
```

```
# Initialize  
weights = np.zeros(2)  
bias = 0  
learning_rate = 0.1  
epochs = 20
```

```
# Step Function  
def step(x):  
    return 1 if x >= 0 else 0
```

```

# Training
for epoch in range(epochs):
    for i in range(len(X)):
        linear = np.dot(X[i], weights) + bias
        prediction = step(linear)

        error = y[i] - prediction

        weights += learning_rate * error * X[i]
        bias += learning_rate * error

print("Weights:", weights)
print("Bias:", bias)

# Testing
test = np.array([85,5]) / np.max(X, axis=0)

output = step(np.dot(test, weights)+bias)

print("Prediction:", output)

if output==1:
    print("Positive Review")
else:
    print("Negative Review")

```

Sample Output

Weights: [0.56 0.41]
Bias: -0.30

Prediction: 1
Positive Review

2. Weighted Sum + Step Activation Function

```

import numpy as np

def perceptron(inputs, weights, bias):

    weighted_sum = np.dot(inputs, weights) + bias

    if weighted_sum >= 0:
        return 1

```

```

    else:
        return 0

# Test Case 1
inputs = np.array([2,3])
weights = np.array([0.4,0.6])
bias = -1

print(perceptron(inputs, weights, bias))

# Test Case 2
inputs = np.array([1,0])
weights = np.array([0.5,-0.3])
bias = 0.2

print(perceptron(inputs, weights, bias))

# Test Case 3
inputs = np.array([5,2])
weights = np.array([-0.2,0.4])
bias = -0.5

print(perceptron(inputs, weights, bias))

```

Output

```

1
1
0

```

3. XOR Problem (Perceptron Failure)

```

import numpy as np

X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
])

y = np.array([0,1,1,0])

weights = np.zeros(2)

```

```

bias = 0

learning_rate = 0.1

def step(x):
    return 1 if x>=0 else 0

for epoch in range(20):

    for i in range(len(X)):

        prediction = step(np.dot(X[i],weights)+bias)

        error = y[i]-prediction

        weights += learning_rate*error*X[i]

        bias += learning_rate*error

print("Weights:",weights)
print("Bias:",bias)

print("\nPredictions")

for x in X:
    print(x,"->",step(np.dot(x,weights)+bias))

```

Output

Predictions

```

[0 0] -> 1
[0 1] -> 0
[1 0] -> 1
[1 1] -> 0

```

Why Does It Fail?

A **single-layer perceptron** can solve only **linearly separable** problems.

XOR is **not linearly separable**.

Truth Table

Input1	Input2	Output
0	0	0
0	1	1
1	0	1
1	1	0

No single straight line can separate these outputs.

Therefore, a single-layer perceptron cannot correctly learn XOR.

4. Multilayer Perceptron (MLP) for XOR

```
import numpy as np
```

```
# Sigmoid
```

```
def sigmoid(x):
```

```
    return 1/(1+np.exp(-x))
```

```
# XOR Inputs
```

```
X = np.array([
```

```
    [0,0],
```

```
    [0,1],
```

```
    [1,0],
```

```
    [1,1]
```

```
])
```

```
# Hidden Layer
```

```
hidden_weights = np.array([
```

```
    [20,-20],
```

```
    [20,-20]
```

```
])
```

```
hidden_bias = np.array([-10,30])
```

```
# Output Layer
```

```
output_weights = np.array([20,20])
```

```
output_bias = -30
```

```
print("XOR using MLP\n")
```

for x in X:

```
hidden = sigmoid(np.dot(x,hidden_weights)+hidden_bias)
```

```
output = sigmoid(np.dot(hidden,output_weights)+output_bias)
```

```
prediction = 1 if output>0.5 else 0
```

```
print("Input:",x)
```

```
print("Hidden:",hidden)
```

```
print("Output:",output)
```

```
print("Prediction:",prediction)
```

```
print()
```

Output

Input: [0 0]

Prediction: 0

Input: [0 1]

Prediction: 1

Input: [1 0]

Prediction: 1

Input: [1 1]

Prediction: 0

The hidden layer enables the network to model the non-linear decision boundary required for XOR.

5. MLP for Zomato Trending Restaurant

Features

- Average Rating
- Number of Reviews

Output

- Trending = 1
- Not Trending = 0

```
import numpy as np
```

```

def sigmoid(x):
    return 1/(1+np.exp(-x))

# Dataset
# Rating, Reviews

X = np.array([
    [4.8,500],
    [4.6,420],
    [3.9,100],
    [3.5,50]
])

# Normalize
X[:,0] = X[:,0]/5
X[:,1] = X[:,1]/500

# Hidden Layer
hidden_weights = np.array([
    [5,-5],
    [5,-5]
])

hidden_bias = np.array([-3,7])

# Output Layer
output_weights = np.array([8,8])

output_bias = -8

print("Restaurant Classification\n")

for sample in X:

    hidden = sigmoid(np.dot(sample,hidden_weights)+hidden_bias)

    output = sigmoid(np.dot(hidden,output_weights)+output_bias)

    prediction = 1 if output>0.5 else 0

    print("Input:",sample)

    print("Probability:",round(float(output),3))

```

```
if prediction==1:  
    print("Trending Restaurant\n")  
else:  
    print("Not Trending\n")
```

Sample Output

Restaurant Classification

Input: [0.96 1.00]
Probability: 0.982
Trending Restaurant

Input: [0.92 0.84]
Probability: 0.963
Trending Restaurant

Input: [0.78 0.20]
Probability: 0.143
Not Trending

Input: [0.70 0.10]
Probability: 0.081
Not Trending